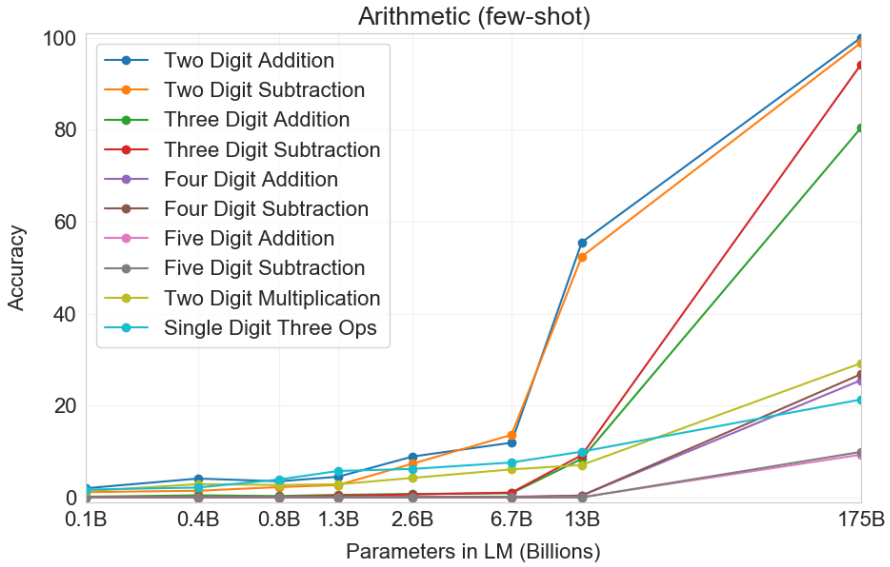

Figure 3.10: Results on all 10 arithmetic tasks in the few-shot settings for models of different sizes. There is a significant jump from the second largest model (GPT-3 13B) to the largest model (GPT-3 175), with the latter being able to reliably accurate 2 digit arithmetic, usually accurate 3 digit arithmetic, and correct answers a significant fraction of the time on 4-5 digit arithmetic, 2 digit multiplication, and compound operations. Results for one-shot and zero-shot are shown in the appendix.

- **3 digit subtraction (3D-)** – Same as 2 digit subtraction, except numbers are uniformly sampled from $[0, 1000)$.
- **4 digit addition (4D+)** – Same as 3 digit addition, except uniformly sampled from $[0, 10000)$.
- **4 digit subtraction (4D-)** – Same as 3 digit subtraction, except uniformly sampled from $[0, 10000)$.
- **5 digit addition (5D+)** – Same as 3 digit addition, except uniformly sampled from $[0, 100000)$.
- **5 digit subtraction (5D-)** – Same as 3 digit subtraction, except uniformly sampled from $[0, 100000)$.
- **2 digit multiplication (2Dx)** – The model is asked to multiply two integers sampled uniformly from $[0, 100)$, e.g. "Q: What is 24 times 42? A: 1008".
- **One-digit composite (1DC)** – The model is asked to perform a composite operation on three 1 digit numbers, with parentheses around the last two. For example, "Q: What is 6+(4*8)? A: 38". The three 1 digit numbers are selected uniformly on $[0, 10)$ and the operations are selected uniformly from {+,-,*}.

In all 10 tasks the model must generate the correct answer exactly. For each task we generate a dataset of 2,000 random instances of the task and evaluate all models on those instances.

First we evaluate GPT-3 in the few-shot setting, for which results are shown in Figure 3.10. On addition and subtraction, GPT-3 displays strong proficiency when the number of digits is small, achieving 100% accuracy on 2 digit addition, 98.9% at 2 digit subtraction, 80.2% at 3 digit addition, and 94.2% at 3-digit subtraction. Performance decreases as the number of digits increases, but GPT-3 still achieves 25-26% accuracy on four digit operations and 9-10% accuracy on five digit operations, suggesting at least some capacity to generalize to larger numbers of digits. GPT-3 also achieves 29.2% accuracy at 2 digit multiplication, an especially computationally intensive operation. Finally, GPT-3 achieves 21.3% accuracy at single digit combined operations (for example, 9*(7+5)), suggesting that it has some robustness beyond just single operations.

As Figure 3.10 makes clear, small models do poorly on all of these tasks – even the 13 billion parameter model (the second largest after the 175 billion full GPT-3) can solve 2 digit addition and subtraction only half the time, and all other operations less than 10% of the time.

One-shot and zero-shot performance are somewhat degraded relative to few-shot performance, suggesting that adaptation to the task (or at the very least recognition of the task) is important to performing these computations correctly. Nevertheless, one-shot performance is still quite strong, and even zero-shot performance of the full GPT-3 significantly

Setting	2D+	2D-	3D+	3D-	4D+	4D-	5D+	5D-	2Dx	1DC
GPT-3 Zero-shot	76.9	58.0	34.2	48.3	4.0	7.5	0.7	0.8	19.8	9.8
GPT-3 One-shot	99.6	86.4	65.5	78.7	14.0	14.0	3.5	3.8	27.4	14.3
GPT-3 Few-shot	100.0	98.9	80.4	94.2	25.5	26.8	9.3	9.9	29.2	21.3

Table 3.9: Results on basic arithmetic tasks for GPT-3 175B. {2,3,4,5}D{+,-} is 2, 3, 4, and 5 digit addition or subtraction, 2Dx is 2 digit multiplication. 1DC is 1 digit composite operations. Results become progressively stronger moving from the zero-shot to one-shot to few-shot setting, but even the zero-shot shows significant arithmetic abilities.

Setting	CL	A1	A2	RI	RW
GPT-3 Zero-shot	3.66	2.28	8.91	8.26	0.09
GPT-3 One-shot	21.7	8.62	25.9	45.4	0.48
GPT-3 Few-shot	37.9	15.1	39.7	67.2	0.44

Table 3.10: GPT-3 175B performance on various word unscrambling and word manipulation tasks, in zero-, one-, and few-shot settings. CL is "cycle letters in word", A1 is anagrams of but the first and last letters, A2 is anagrams of all but the first and last two letters, RI is "Random insertion in word", RW is "reversed words".

outperforms few-shot learning for all smaller models. All three settings for the full GPT-3 are shown in Table 3.9, and model capacity scaling for all three settings is shown in Appendix H.

To spot-check whether the model is simply memorizing specific arithmetic problems, we took the 3-digit arithmetic problems in our test set and searched for them in our training data in both the forms "<NUM1> + <NUM2> =" and "<NUM1> plus <NUM2>". Out of 2,000 addition problems we found only 17 matches (0.8%) and out of 2,000 subtraction problems we found only 2 matches (0.1%), suggesting that only a trivial fraction of the correct answers could have been memorized. In addition, inspection of incorrect answers reveals that the model often makes mistakes such as not carrying a "1", suggesting it is actually attempting to perform the relevant computation rather than memorizing a table.

Overall, GPT-3 displays reasonable proficiency at moderately complex arithmetic in few-shot, one-shot, and even zero-shot settings.

3.9.2 Word Scrambling and Manipulation Tasks

To test GPT-3's ability to learn novel symbolic manipulations from a few examples, we designed a small battery of 5 "character manipulation" tasks. Each task involves giving the model a word distorted by some combination of scrambling, addition, or deletion of characters, and asking it to recover the original word. The 5 tasks are:

- **Cycle letters in word (CL)** – The model is given a word with its letters cycled, then the "=" symbol, and is expected to generate the original word. For example, it might be given "lyinevitab" and should output "inevitably".
- **Anagrams of all but first and last characters (A1)** – The model is given a word where every letter except the first and last have been scrambled randomly, and must output the original word. Example: criroptuon = corruption.
- **Anagrams of all but first and last 2 characters (A2)** – The model is given a word where every letter except the first 2 and last 2 have been scrambled randomly, and must recover the original word. Example: opoepnnt → opponent.
- **Random insertion in word (RI)** – A random punctuation or space character is inserted between each letter of a word, and the model must output the original word. Example: s.u!c/c!e.s s i/o/n = succession.
- **Reversed words (RW)** – The model is given a word spelled backwards, and must output the original word. Example: stcejbo → objects.

For each task we generate 10,000 examples, which we chose to be the top 10,000 most frequent words as measured by [Nor09] of length more than 4 characters and less than 15 characters. The few-shot results are shown in Figure 3.11. Task performance tends to grow smoothly with model size, with the full GPT-3 model achieving 66.9% on removing

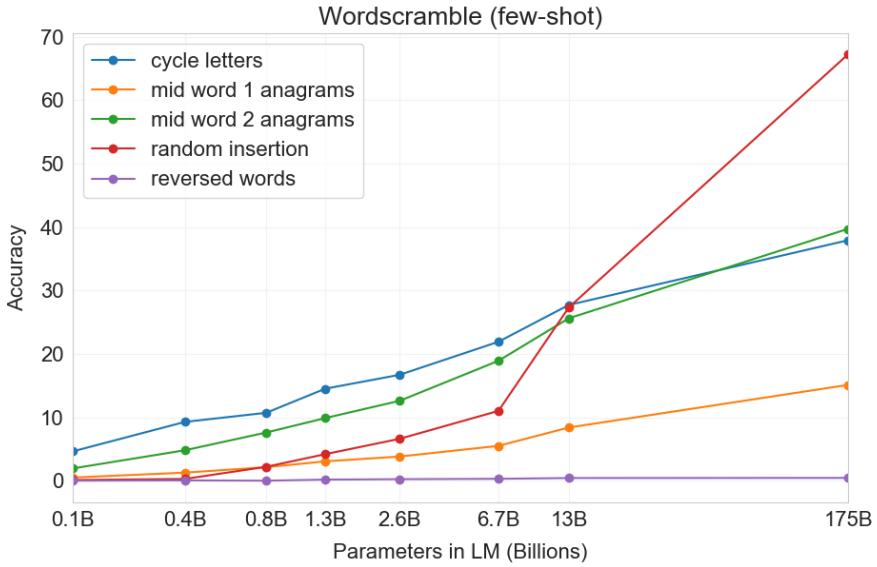

Figure 3.11: Few-shot performance on the five word scrambling tasks for different sizes of model. There is generally smooth improvement with model size although the random insertion task shows an upward slope of improvement with the 175B model solving the task the majority of the time. Scaling of one-shot and zero-shot performance is shown in the appendix. All tasks are done with $K = 100$.

random insertions, 38.6% on cycling letters, 40.2% on the easier anagram task, and 15.1% on the more difficult anagram task (where only the first and last letters are held fixed). None of the models can reverse the letters in a word.

In the one-shot setting, performance is significantly weaker (dropping by half or more), and in the zero-shot setting the model can rarely perform any of the tasks (Table 3.10). This suggests that the model really does appear to learn these tasks at test time, as the model cannot perform them zero-shot and their artificial nature makes them unlikely to appear in the pre-training data (although we cannot confirm this with certainty).

We can further quantify performance by plotting "in-context learning curves", which show task performance as a function of the number of in-context examples. We show in-context learning curves for the Symbol Insertion task in Figure 1.2. We can see that larger models are able to make increasingly effective use of in-context information, including both task examples and natural language task descriptions.

Finally, it is worth adding that solving these tasks requires character-level manipulations, whereas our BPE encoding operates on significant fractions of a word (on average ~ 0.7 words per token), so from the LM's perspective succeeding at these tasks involves not just manipulating BPE tokens but understanding and pulling apart their substructure. Also, CL, A1, and A2 are not bijective (that is, the unscrambled word is not a deterministic function of the scrambled word), requiring the model to perform some search to find the correct unscrambling. Thus, the skills involved appear to require non-trivial pattern-matching and computation.

3.9.3 SAT Analogies

To test GPT-3 on another task that is somewhat unusual relative to the typical distribution of text, we collected a set of 374 "SAT analogy" problems [TLBS03]. Analogies are a style of multiple choice question that constituted a section of the SAT college entrance exam before 2005. A typical example is "audacious is to boldness as (a) sanctimonious is to hypocrisy, (b) anonymous is to identity, (c) remorseful is to misdeed, (d) deleterious is to result, (e) impressionable is to temptation". The student is expected to choose which of the five word pairs has the same relationship as the original word pair; in this example the answer is "sanctimonious is to hypocrisy". On this task GPT-3 achieves 65.2% in the few-shot setting, 59.1% in the one-shot setting, and 53.7% in the zero-shot setting, whereas the average score among college applicants was 57% [TL05] (random guessing yields 20%). As shown in Figure 3.12, the results improve with scale, with the the full 175 billion model improving by over 10% compared to the 13 billion parameter model.